DweLink
PHARMACY & HEALTH PLATFORM

PROJET ARCHITECTURE MICROSERVICES

DweLink

Le lien entre patients, donateurs et pharmacies

Plateforme de gestion des médicaments, des pharmacies et du don solidaire · Spring Boot + Angular

Module AWD · Validation 1 | Classe 4DAS13 · Groupe 6 | 12-11-2026



Notre équipe



Hichem Jemail



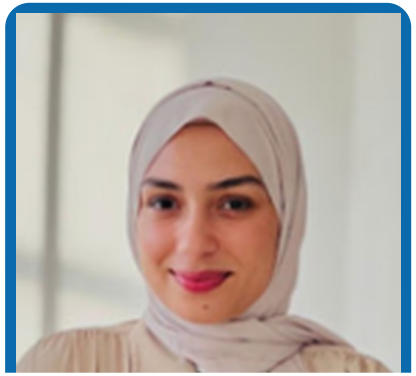
Mohamed Yassine Ben Aïssa



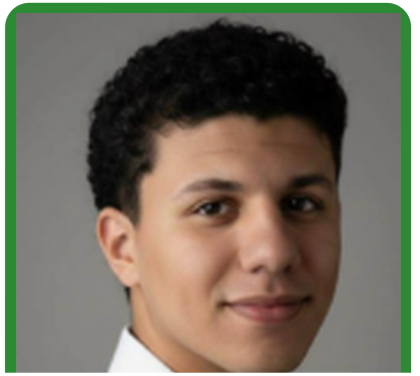
Wejdene Bouali



Rayen Sassi



Roua Selmi



Mohamed Yassine Azzouz

SOMMAIRE

Plan de la présentation

01 **Idée et problématique**
Le constat et la question

02 **Fonctionnalités**
Quatre acteurs, cas d'utilisation

03 **Architecture**
Composants et communications

04 **Microservices et données**
Sept services, sept bases

05 **Technologies**
Les choix techniques

06 **Organisation**
Dépôt Git et risques

07 **Les microservices en détail**
Diagrammes de classes et API

? **Questions**
Échange avec le jury

01

Mieux accéder aux médicaments

L'idée

Une plateforme pour patients, pharmaciens et donateurs

Problématique

Comment faciliter l'accès aux médicaments, la gestion du stock et le don solidaire sur une seule plateforme ?



Information

Où trouver mon médicament ?

Gestion

Stock et péremptions mal suivis

Gaspillage

Des boîtes utilisables jetées

CONCEPTION · RÉPONSE

01

Une solution en sept microservices



OBJECTIFS

- Trouver et réserver un médicament dans une pharmacie proche
- Suivre le stock, les lots et les dates de péremption
- Organiser le don gratuit, contrôlé par un pharmacien
- Valider les pharmaciens et les pharmacies

02

Principales fonctionnalités par acteur

Patient

- Rechercher un médicament
- Trouver une pharmacie proche
- Réserver un médicament
- Suivre ou annuler ses réservations

Donateur

- Proposer un don
- Choisir une pharmacie
- Déposer le médicament
- Suivre ou annuler ses dons

Pharmacien

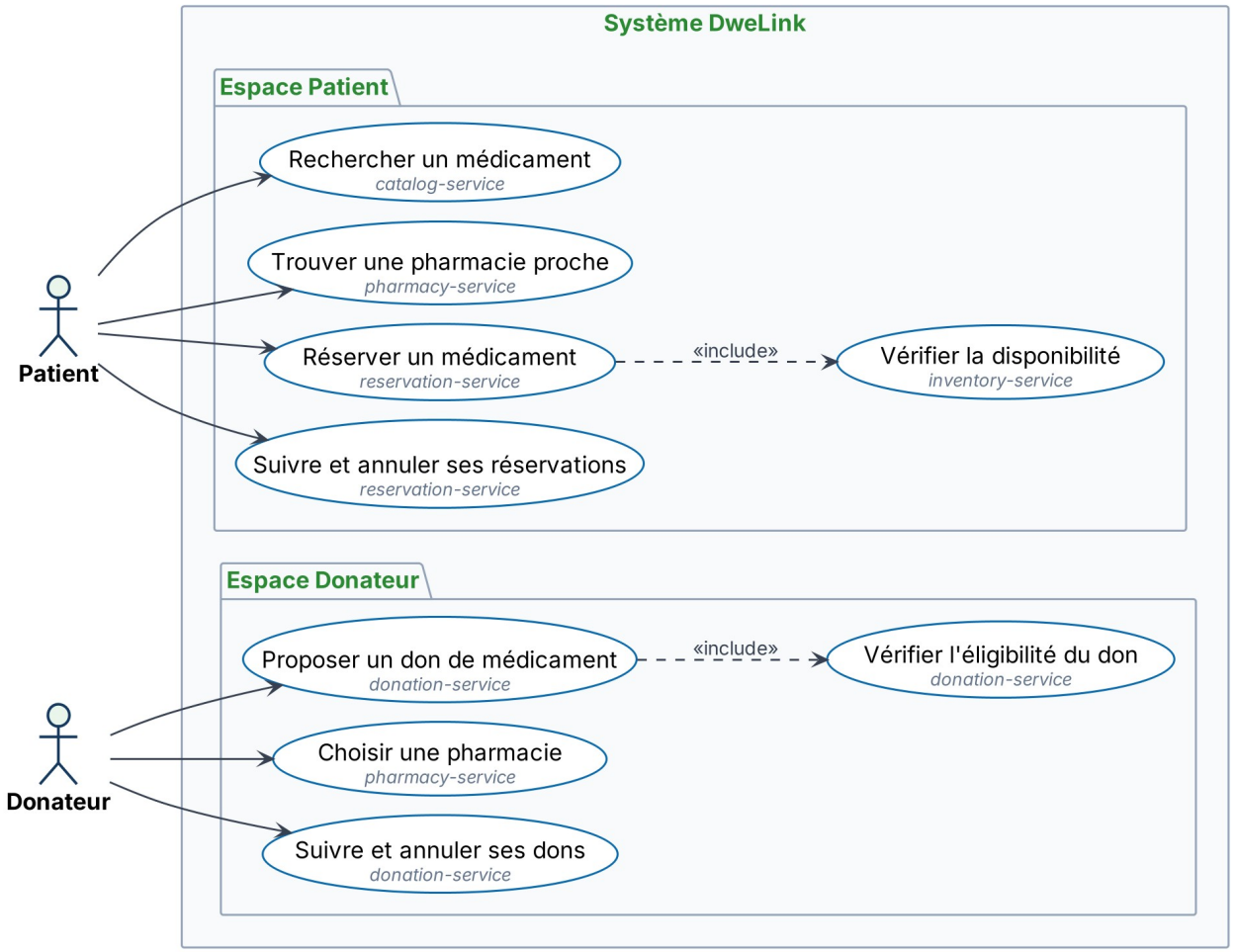
- Gérer sa pharmacie et ses membres
- Gérer le stock et les lots
- Traiter les réservations
- Contrôler et remettre les dons

Administrateur

- Valider les pharmaciens
- Valider les pharmacies
- Gérer le catalogue
- Gérer les comptes

02

Patient et Donateur



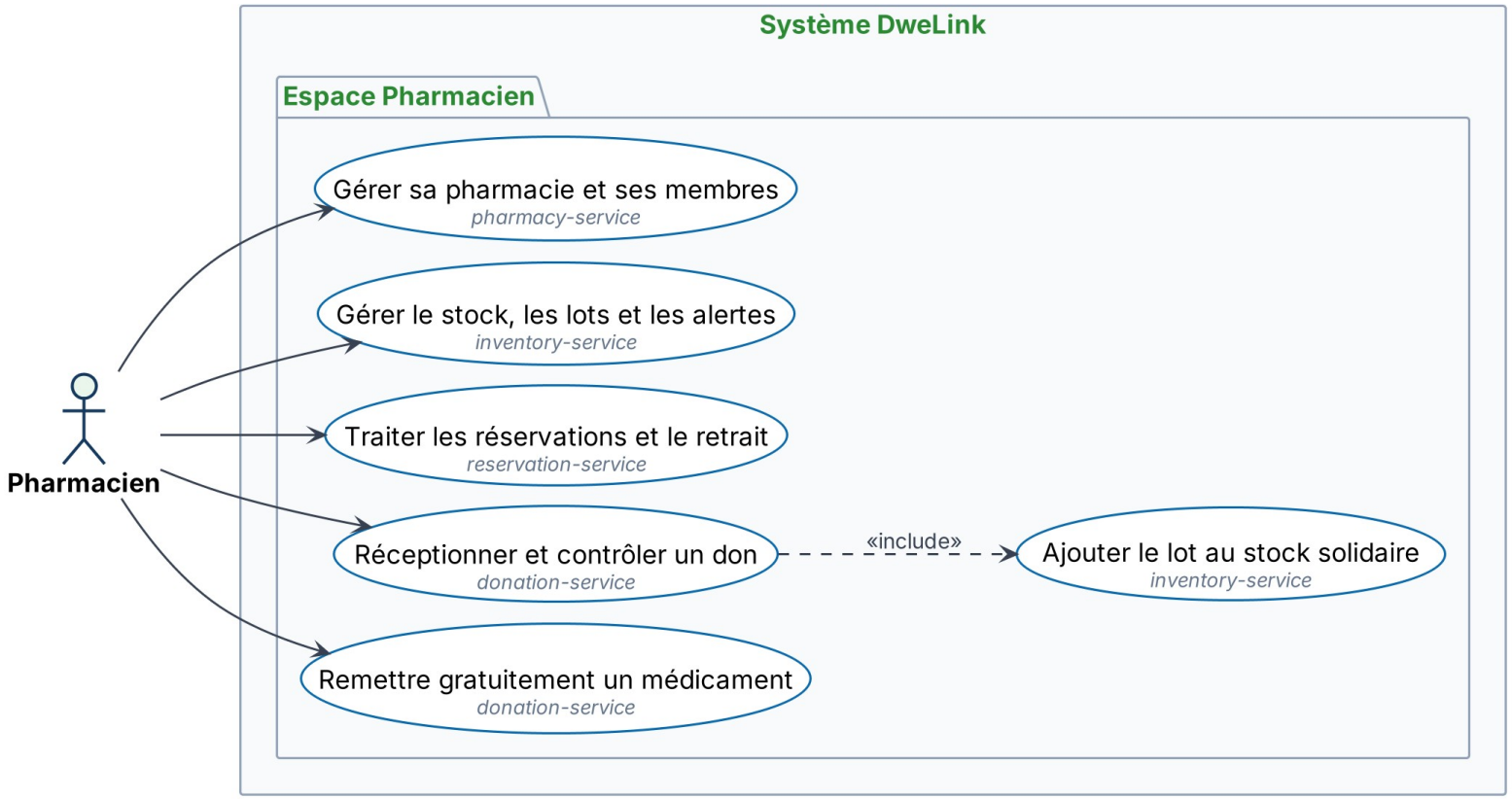
Rechercher · Réserver

Réserver → «include» disponibilité

Proposer un don → «include» éligibilité

02

Pharmacien

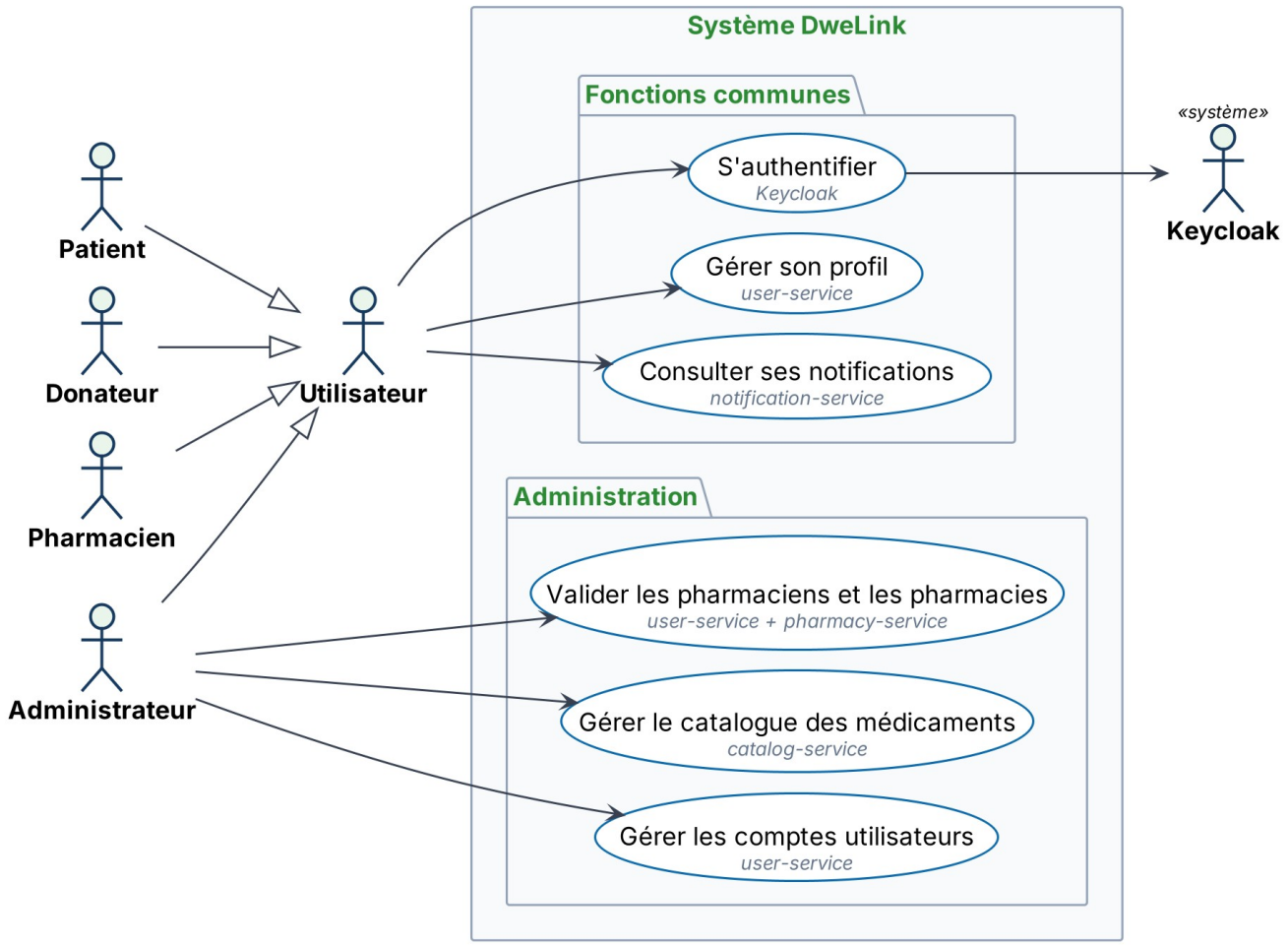


Stock · Lots · Alertes

Contrôler un don → «include» stock solidaire

02

Administrateur et fonctions communes



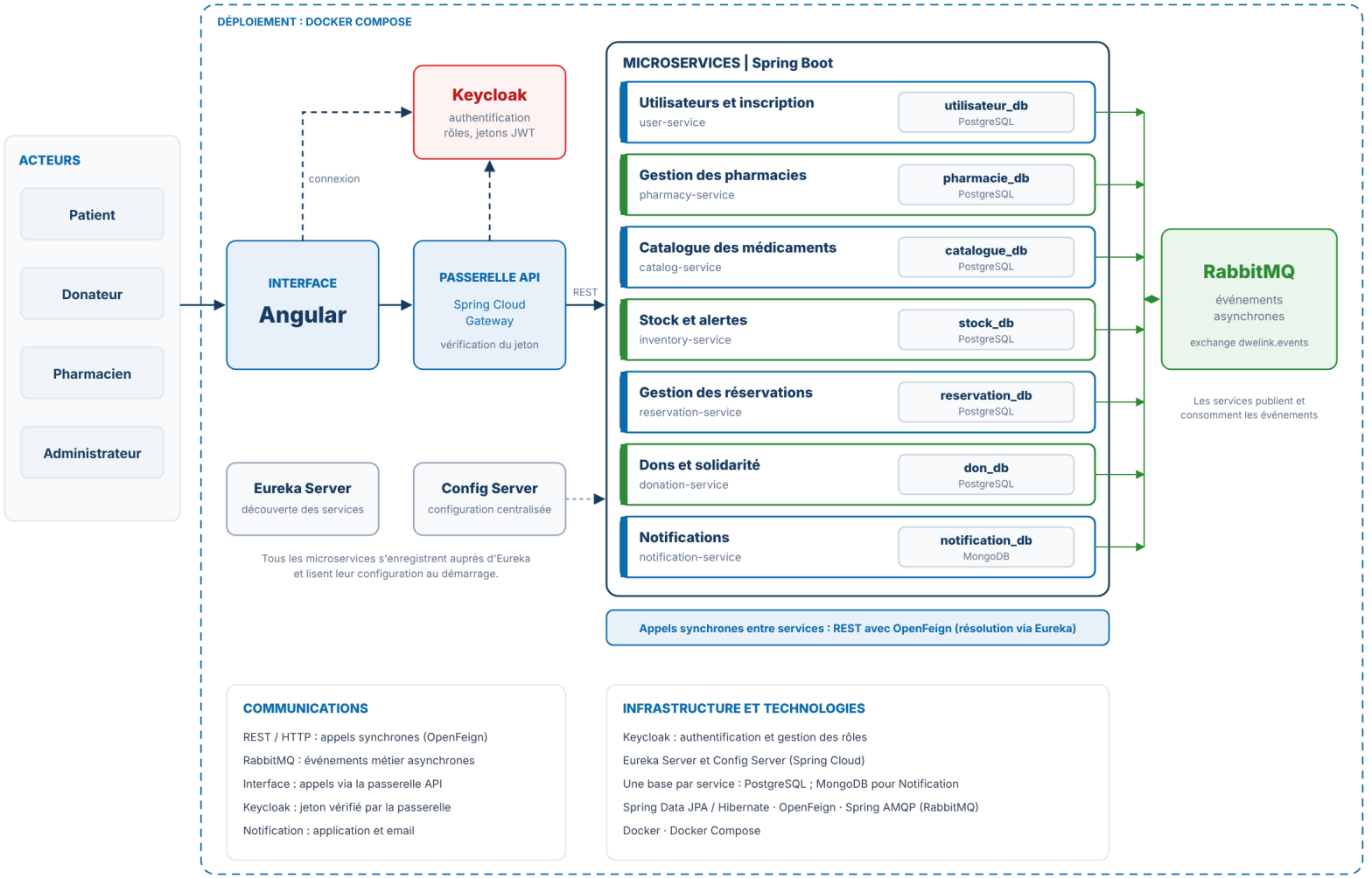
4 acteurs héritent de « Utilisateur »

Authentification par Keycloak

Validation · Catalogue · Comptes

03

Architecture microservices



Passerelle API
Entrée unique

Keycloak
Sécurité

Eureka · Config
Découverte · Configuration

RabbitMQ
Événements

04

Responsabilités et bases de données

01 · Utilisateurs et inscription
`user-service`

Profils · Comptes · Validation des pharmaciens

`utilisateur_db` PostgreSQL

02 · Gestion des pharmacies
`pharmacy-service`

Pharmacies · Horaires · Gardes · Membres

`pharmacie_db` PostgreSQL

03 · Catalogue des médicaments
`catalog-service`

Médicaments · Compositions · Équivalents

`catalogue_db` PostgreSQL

04 · Stock et alertes
`inventory-service`

Lots · Péremption · Alertes

`stock_db` PostgreSQL

05 · Gestion des réservations
`reservation-service`

Réservation · Code de retrait · Expiration

`reservation_db` PostgreSQL

06 · Dons et solidarité
`donation-service`

Don · Contrôle · Remise solidaire

`don_db` PostgreSQL

07 · Notifications
`notification-service`

Application · Email · Préférences

`notification_db` MongoDB

04

Appel direct : demande puis réponse

OpenFeign · REST · HTTP

Réservations
service appelant

① **Demande**

`POST /api/stock/reservations`

② **Réponse · 201 Created**

Stock
service appelé

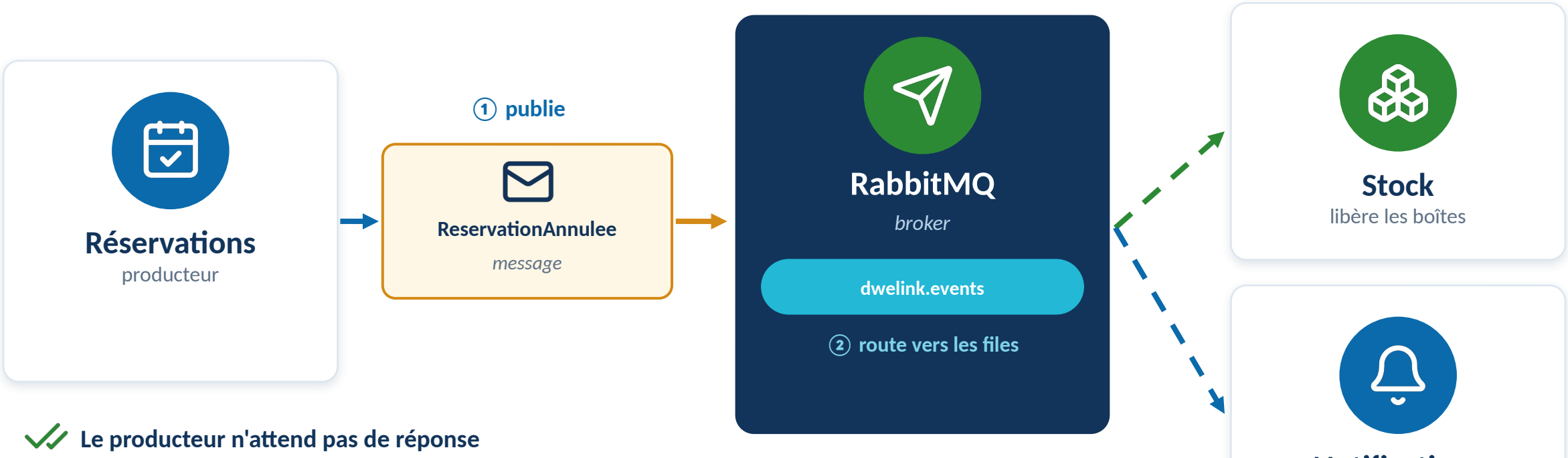
L'appelant attend

AUTRES APPELS SYNCHRONES

- Pharmacies → Utilisateurs
- Stock → Catalogue
- Stock → Pharmacies
- Réservations → Catalogue
- Réservations → Pharmacies
- Dons → Catalogue
- Dons → Pharmacies
- Notifications → Utilisateurs

04

Événements via RabbitMQ, sans attente



Le producteur n'attend pas de réponse

21 ÉVÉNEMENTS PUBLIÉS

Utilisateurs · 3 | Pharmacies · 3 | Catalogue · 1 | Stock · 3 | Réservations · 6 | Dons · 5

05

Technologies choisies

INTERFACE

Angular

4 espaces utilisateurs

SERVEUR

Spring Boot

Gateway · Eureka · Config · OpenFeign

SÉCURITÉ

Keycloak

Connexion · Rôles · Jetons

DONNÉES

PostgreSQL · MongoDB

Une base par service · JPA

MESSAGERIE

RabbitMQ

Événements asynchrones

DÉPLOIEMENT

Docker · Git

Compose · GitHub · JUnit · Postman

06

Organisation du dépôt

```
dwelink/
├── Backend/
│   ├── gateway-service/
│   ├── discovery-service/  (Eureka)
│   ├── config-service/
│   ├── user-service/   ├── pharmacy-service/
│   ├── catalog-service/   ├── inventory-service/
│   ├── reservation-service/
│   ├── donation-service/
│   └── notification-service/
├── Frontend/dwelink-web/  (Angular)
├── Documentation/  (Presentation, Diagrams)
├── docker-compose.yml
└── README.md
```

Une branche par fonctionnalité
Fusion par pull request relue par un autre membre

Commits personnels
Chaque membre contribue régulièrement à son service

Suivi des tâches
Tableau GitHub Projects : à faire, en cours, terminé

Lancement unique
docker compose up démarre tout le projet

06

Gestion des risques

Retard dans le travail

Développer d'abord les fonctions principales

Services qui communiquent mal

Tester les échanges au fur et à mesure

Outils difficiles à configurer

Installer Keycloak et RabbitMQ dès le début

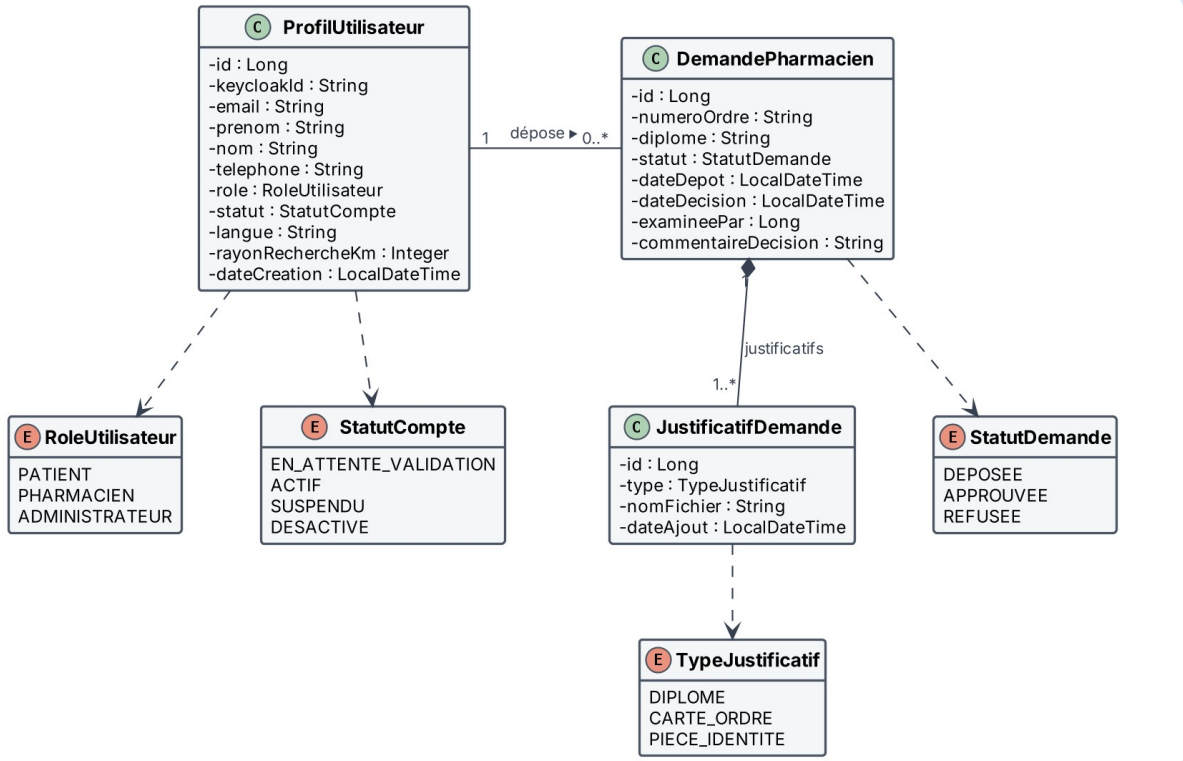
Don de médicament non conforme

Chaque don est contrôlé par un pharmacien

Utilisateurs et inscription

utilisateur_db · PostgreSQL

01



Méthode	Endpoint	Rôle	Codes
POST	`/api/utilisateurs/moi`	Utilisateur	201, 400, 409
GET	`/api/utilisateurs/moi`	Utilisateur	200, 404
PATCH	`/api/utilisateurs/moi`	Utilisateur	200, 400
POST	`/api/demandes-pharmacien`	Utilisateur	201, 400, 409
POST	`/api/demandes-pharmacien/{id}/justificatifs`	Utilisateur	201, 400, 404
GET	`/api/demandes-pharmacien?statut=`	Administrateur	200, 403
PATCH	`/api/demandes-pharmacien/{id}/decision`	Administrateur	200, 403, 404, 409, 503
PATCH	`/api/utilisateurs/{id}/statut`	Administrateur	200, 403, 404
GET	`/api/utilisateurs/{id}`	*Services internes*	200, 404
GET	`/api/utilisateurs/{id}/contact`	*Services internes*	200, 404

SYNCHRONE · REST
Appelle : Keycloak
Appelé par : Pharmacies, Notifications

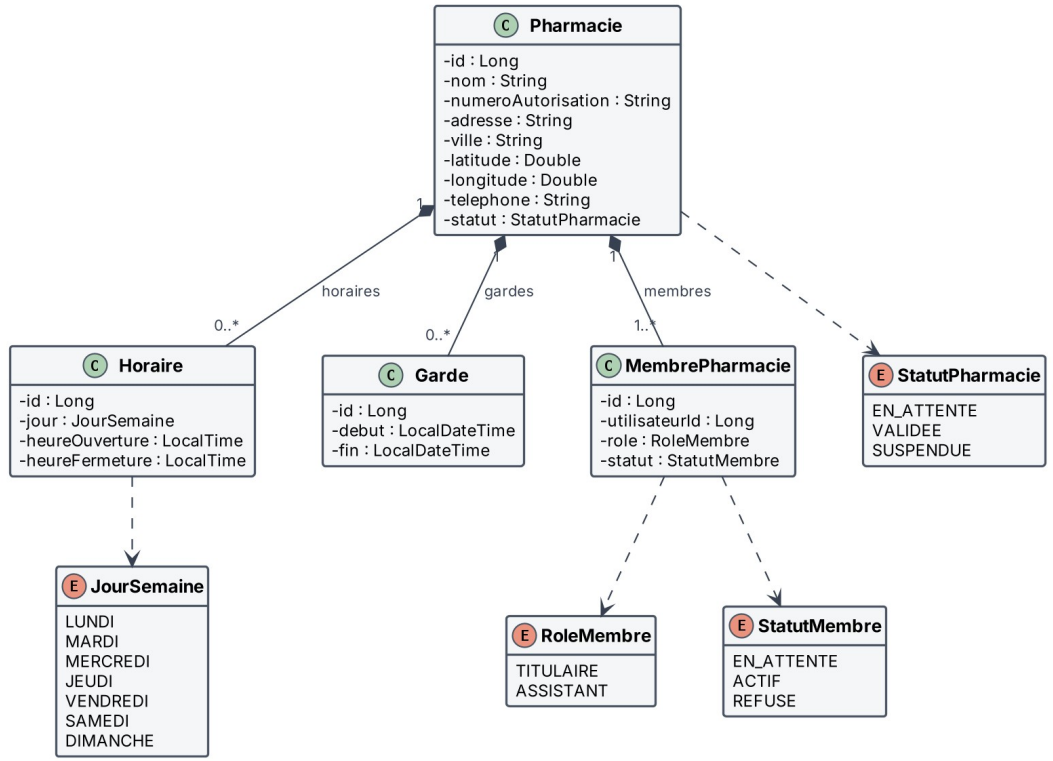
ASYNCHRONE · RABBITMQ
Émet : DemandePharmacienDeposee, DemandePharmacienDecidee, StatutCompteModifie
Consomme : aucun événement

RÉSILIENCE
Keycloak ne répond pas : erreur 503, la demande reste en attente

Gestion des pharmacies

pharmacie_db · PostgreSQL

02



Méthode	Endpoint	Rôle	Codes
POST	/api/pharmacies	Pharmacien	201, 400, 409
GET	/api/pharmacies/{id}	Utilisateur	200, 404
PUT	/api/pharmacies/{id}	Pharmacien (titulaire)	200, 400, 403, 404
GET	/api/pharmacies/proches?lat=&lng=&rayonKm=	Utilisateur	200, 400
GET	/api/pharmacies?ville=&deGarde=	Utilisateur	200
PATCH	/api/pharmacies/{id}/statut	Administrateur	200, 403, 404, 409
PUT	/api/pharmacies/{id}/horaires	Pharmacien (titulaire)	200, 400, 403
POST	/api/pharmacies/{id}/gardes	Pharmacien (titulaire)	201, 400, 403, 409
POST	/api/pharmacies/{id}/membres	Pharmacien	201, 403, 409
PATCH	/api/pharmacies/{id}/membres/{membreId}	Pharmacien (titulaire)	200, 403, 404, 409
DELETE	/api/pharmacies/{id}/membres/{membreId}	Pharmacien (titulaire)	204, 403, 404
GET	/api/pharmacies/{id}/membres/verifier?utilisateurId=	*Services internes*	200, 404

SYNCHRONE · REST
Appelle : Utilisateurs
Appelé par : Stock, Réservations, Dons, Interface Angular

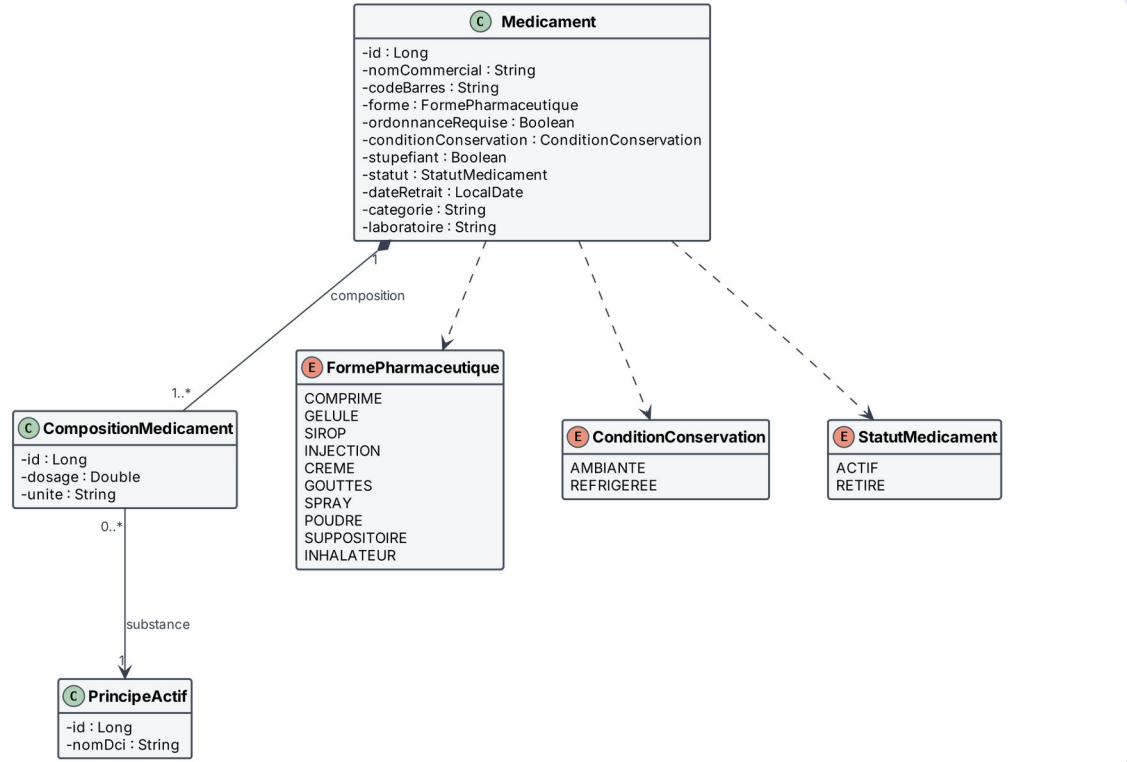
ASYNCHRONE · RABBITMQ
Émet : StatutPharmacieModifie, DemandeRattachementCreee, MembresPharmacieModifies
Consomme : StatutCompteModifie

RÉSILIENCE
user-service ne répond pas : erreur 503, rien n'est créé

Catalogue des médicaments

catalogue_db · PostgreSQL

03



Méthode	Endpoint	Rôle	Codes
GET	`/api/medicaments?q=&categorie=&laboratoire=`	Utilisateur	200
GET	`/api/medicaments/{id}`	Utilisateur	200, 404
GET	`/api/medicaments/{id}/equivalents`	Utilisateur	200, 404
POST	`/api/medicaments`	Administrateur	201, 400, 403, 409
PUT	`/api/medicaments/{id}`	Administrateur	200, 400, 403, 404
PATCH	`/api/medicaments/{id}/retrait`	Administrateur	200, 403, 404, 409
POST	`/api/medicaments/{id}/compositions`	Administrateur	201, 400, 403, 404
DELETE	`/api/medicaments/{id}/compositions/{compositionId}`	Administrateur	204, 403, 404, 409
GET	`/api/principes-actifs?q=`	Utilisateur	200
POST	`/api/principes-actifs`	Administrateur	201, 400, 403, 409

SYNCHRONE · REST
Appelle : aucun service
Appelé par : Stock, Réservations, Dons, Interface Angular

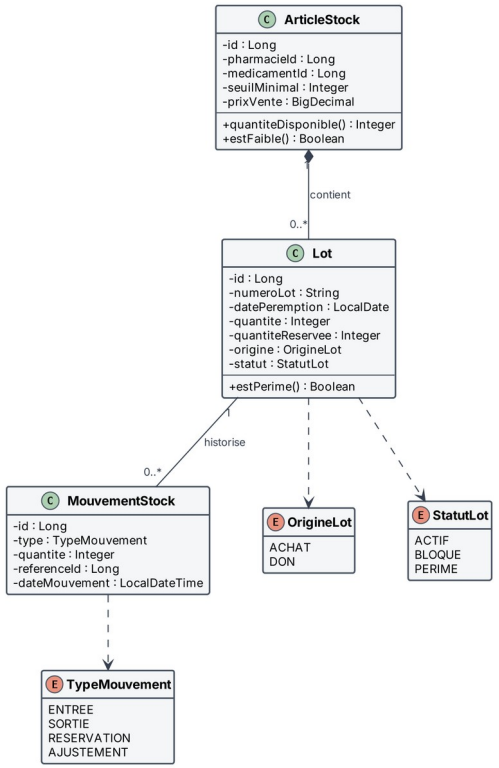
ASYNCHRONE · RABBITMQ
Émet : MedicamentRetire
Consomme : aucun événement

RÉSILIENCE
Le catalogue n'appelle aucun autre service

Stock et alertes

stock_db · PostgreSQL

04



Méthode	Endpoint	Rôle	Codes
POST	/api/articles-stock	Pharmacien	201, 400, 403, 409
GET	/api/articles-stock?pharmacieId=	Pharmacien	200, 403
PATCH	/api/articles-stock/{id}	Pharmacien	200, 400, 403, 404
POST	/api/articles-stock/{id}/lots	Pharmacien	201, 400, 403, 409
PATCH	/api/lots/{id}/statut	Pharmacien	200, 403, 404
POST	/api/articles-stock/{id}/mouvements	Pharmacien	201, 400, 403, 409
GET	/api/articles-stock/{id}/mouvements	Pharmacien	200, 403
GET	/api/disponibilite?medicamentId=&lat=&lng=	Utilisateur	200, 400
POST	/api/stock/reservations	*Services internes*	201, 409

SYNCHRONE · REST
Appelle : Catalogue, Pharmacies
Appelé par : Réservations, Interface Angular

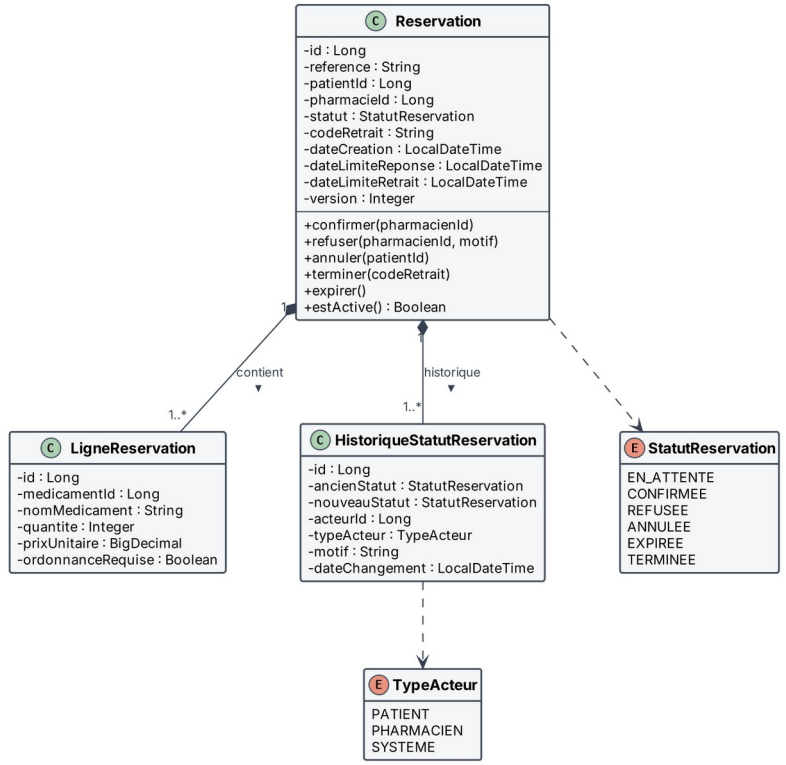
ASYNCHRONE · RABBITMQ
Émet : StockFaible, RuptureStock, PeremptionProche
Consomme : ReservationRefusee / Annulee / Expiree, ReservationTerminee, DonRecu / RemiseEffectuee, MedicamentRetire, StatutPharmacieModifie

RÉSILIENCE
Catalogue ou Pharmacies ne répond pas : erreur 503, le stock déjà enregistré reste utilisable

Gestion des réservations

reservation_db · PostgreSQL

05



Méthode	Endpoint	Rôle	Codes
POST	/api/reservations	Patient	201, 400, 404, 409, 503
GET	/api/reservations/{id}	Patient · Pharmacien	200, 403, 404
GET	/api/reservations/mes-reservations?statut=	Patient	200
GET	/api/reservations?pharmacieId=&statut=	Pharmacien	200, 403
PATCH	/api/reservations/{id}/confirmer	Pharmacien	200, 403, 404, 409
PATCH	/api/reservations/{id}/refuser	Pharmacien	200, 400, 403, 404, 409
PATCH	/api/reservations/{id}/annuler	Patient	200, 403, 404, 409
PATCH	/api/reservations/{id}/terminer	Pharmacien	200, 400, 403, 404, 409
GET	/api/reservations/{id}/historique	Patient · Pharmacien	200, 403, 404

SYNCHRONE · REST
Appelle : Catalogue, Pharmacies, Stock
Appelé par : Interface Angular

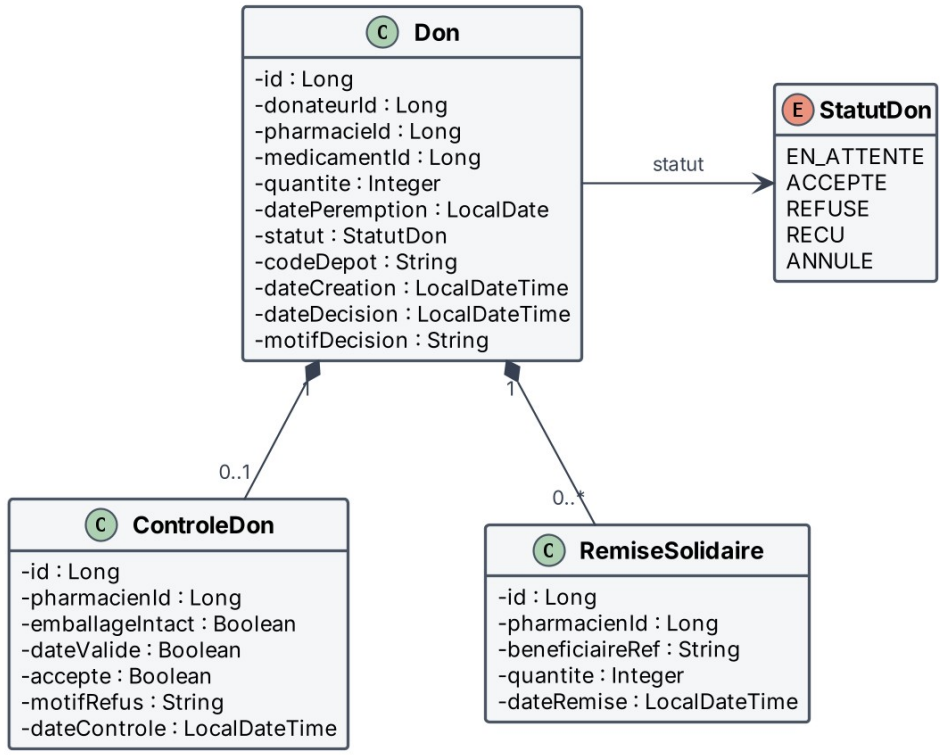
ASYNCHRONE · RABBITMQ
Émet : ReservationCreee, ReservationConfirmee, ReservationRefusee, ReservationAnnulee, ReservationExpiree, ReservationTerminee
Consomme : StatutPharmacieModifie, MedicamentRetire

RÉSILIENCE
Stock ne répond pas : erreur 503, la réservation n'est pas créée

Dons et solidarité

don_db · PostgreSQL

06



Méthode	Endpoint	Rôle	Codes
POST	/api/dons	Donateur	201, 400, 403, 422, 503
GET	/api/dons/{id}	Donateur · Pharmacien	200, 403, 404
GET	/api/dons/mes-dons	Donateur	200, 403
PATCH	/api/dons/{id}/annuler	Donateur	200, 403, 404, 409
GET	/api/dons?pharmacieId=&statut=&page=	Pharmacien	200, 403
PATCH	/api/dons/{id}/decision	Pharmacien	200, 400, 403, 404, 409
POST	/api/dons/{id}/controle	Pharmacien	201, 400, 403, 404, 409
POST	/api/dons/{id}/remises	Pharmacien	201, 400, 403, 404, 409

SYNCHRONE · REST
Appelle : Catalogue, Pharmacies
Appelé par : Interface Angular

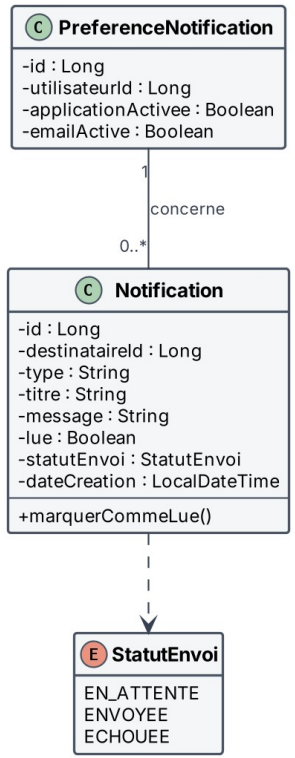
ASYNCHRONE · RABBITMQ
Émet : DonPropose, DonDecide, DonAnnule, DonRecu, RemiseEffectuee
Consomme : MedicamentRetire, StatutPharmacieModifie

RÉSILIENCE
Catalogue ou Pharmacies ne répond pas : erreur 503, le don n'est pas enregistré

Notifications

notification_db · MongoDB

07



Méthode	Endpoint	Rôle	Codes
GET	`/api/notifications?lue=`	Utilisateur	200, 401
GET	`/api/notifications/non-lues/nombre`	Utilisateur	200, 401
PUT	`/api/notifications/{id}/lue`	Utilisateur	200, 403, 404
PUT	`/api/notifications/tout-lu`	Utilisateur	204, 401
PUT	`/api/notifications/preferences`	Utilisateur	200, 400

SYNCHRONE · REST
Appelle : Utilisateurs
Appelé par : Interface Angular

ASYNCHRONE · RABBITMQ
Émet : aucun événement
Consomme : Événements des comptes, Événements des pharmacies, Alertes de stock, Événements des réservations, Événements des dons

RÉSILIENCE
RabbitMQ ne répond pas : les messages attendent et sont traités ensuite

Conclusion

- Une plateforme qui relie patients, pharmaciens et donateurs
- 7 microservices simples et indépendants
- Des échanges directs (REST) et par messages (RabbitMQ)
- Un accès sécurisé avec Keycloak
- Moins de gaspillage grâce au don solidaire



Prochaine étape

Développer, intégrer et tester

CONCLUSION

DweLink en un coup d'œil



Collaboration
Patients ⇄ Pharmacies ⇄ Donateurs

7
microservices

4
acteurs

Sécurisé · Évolutif · Solidaire

DweLink : chaque médicament trouve son patient.

DweLink
PHARMACY & HEALTH PLATFORM

Merci de votre attention

Des questions ?



DweLink · Classe 4DAS13 · Groupe 6 · Module AWD